A Strict Semantic and Certificate Hierarchy for Cost-Aware Cryptographic Verification in Lean 4

Chen Qian

Abstract. Cryptographic formalizations must connect probability semantics, exact resource accounting, asymptotic complexity, oracle interaction, and interactive security without silently changing the computation being certified. This paper presents the current architecture of Crypto, an experimental Lean 4 library organized around one invariant: every public probability distribution erases a single authoritative exact execution, and every complexity statement certifies that same execution. Exact costs range over ordered, possibly noncommutative additive monoids. Typed, result-indexed primitive signatures separate exact handlers from mathematical laws and operation bounds. Programs, oracle computations, and interactive machines then reuse the same randomized writer and path-bound vocabulary. Input-dependent exact bounds are projected through a monotone additive natural-number measure and refined into uniform and polynomial runtime certificates without replacing the underlying run. The UC layer adds typed ports, a FIFO kernel, adaptive corruption, explicit kernel charges, certified fuel, real/ideal worlds, and operational context composition. We describe the dependency discipline, the main proof interfaces, mechanized case studies, and the soundness boundary of the present model.

Keywords: Lean 4 · cryptographic verification · exact cost semantics · probabilistic programs · universal composability.

1 Introduction

Mechanized cryptography needs several descriptions of one algorithm. A proof of correctness observes a probability distribution; an implementation-level argument records which operations were executed; a complexity proof supplies a worst-case bound; and a security definition admits only adversaries carrying an appropriate polynomial-time certificate. If these descriptions are defined independently, they can drift. In particular, a costed interpreter can disagree with the probability game, a "runtime" field can become metadata unrelated to the run it purports to bound, and an interactive security definition can mention protocols or corruption policies that never enter the actual execution.

This paper describes the current architecture of Crypto, an experimental cryptographic verification library implemented in Lean 4 [dMU21] and built on mathlib [mC20]. The architecture is intended to support game-based and universally composable security in a shared executable substrate. Its central invariant is deliberately stronger than a naming convention.

Design principle. Every public probability semantics is obtained by erasing cost from one authoritative exact execution. Every exact, measured, and polynomial complexity result is a proposition or certificate indexed by that same execution.

Four separations make the invariant practical. First, exact execution and cost-erased probability are different views of one joint value–cost distribution. Second, the cost produced on a path and a proof that it is below a budget are different objects. Third, an abstract resource—which may be multidimensional or order-sensitive—is distinct from the natural-number observation used by the existing asymptotic layer. Fourth, a security game is distinct from the certificate that admits a machine as a PPT adversary. These separations run from primitive operations through typed programs and oracles to the interactive-machine kernel.

ElGamal encryption is a representative example. Its encryption algorithm is a typed program whose exact handler samples one scalar, performs two scalar actions, and performs one group operation. Correctness consumes the erased distribution of that program. Efficiency attaches an input-dependent bound and a measured runtime certificate to the same program; it does not restate the algorithm in a second AST. The one-time pad, discrete logarithm, and Decisional Diffie–Hellman examples follow the same boundary.

Contributions. The present implementation provides:

- a strict import hierarchy separating security parameters, probability, exact cost semantics, asymptotics, machine certificates, game-based notions, and UC execution;
- exact randomized resource semantics over an ordered, potentially noncommutative additive monoid, with a single support-wise path-bound predicate and a monotone additive observation into $\mathbb{N}$;
- typed result-indexed primitive signatures whose exact handlers, cost-erased mathematical laws, and upper bounds are separate records;
- a typed program language with one interpreter, an exact structural execution relation, a support-equivalence theorem, and compositional input-dependent bounds;
- a unified dependent machine hierarchy and a closed exact-to-measured- to-polynomial certificate chain, including a single oracle interpreter and certified caller–oracle composition; and
- a typed FIFO UC execution layer with capabilities, adaptive corruption, explicit kernel charging, certified finite fuel, real/ideal wiring, and a context-composition theorem derived from operational simulation.

The goal of the paper is architectural: to state what the current Lean interfaces guarantee, how the certificates connect, and which obligations remain with an instantiation. It does not claim a new cryptographic reduction or a physical CPU-time model. The distinction is made explicit in Section 8.

Positioning. The library shares the broader aim of foundational mechanized cryptography exemplified by SSProve [AHR+21]. Its interactive layer follows the real/ideal quantifier discipline of universal composability [Can01], while its engineering focus differs from systems such as EasyUC [CSV19]: the contribution described here is a single Lean-native exact resource and certificate hierarchy spanning typed programs, adaptive oracles, and an executable UC kernel. A systematic feature or proof engineering comparison is left for later work.

Organization. Section 2 presents the dependency discipline. Section 3 develops exact resources, typed algebras, and programs. Section 4 gives the machine and oracle certificate chains. Section 5 describes the interactive execution stack. Sections 6 and 7 summarize concrete instantiations and mechanized validation, followed by soundness boundaries and future work.

2 Logical Architecture and Dependency Discipline

The Infrastructure tree is organized as a directed acyclic graph rather than a flat collection of utilities. Three independent roots have distinct roles: `Security Parameter` defines `Crypto.SecPar`; `Probability` contains cost-free probability constructions; and `Cost.Model` defines the exact resource interface. In particular, computation modules no longer import the asymptotic layer merely to name a security parameter.

The executable checker `scripts/check_infrastructure_imports.py` parses Infrastructure imports, rejects cycles, and rejects edges that point upward in the declared subsystem order. Aggregation modules may import completed members of their own layer; implementation modules must follow the finer order encoded by the checker. This is a CI-enforced engineering invariant. It is not presented as a metatheorem about Lean's logic.

2.1 Two orthogonal refinement directions

The dependency DAG contains two conceptually different forms of refinement. The semantic direction adds structure to exact execution:

$$\begin{aligned}&\text{resource writer} \longrightarrow \text{typed operation handler}\\ &\quad\longrightarrow \text{program/oracle/kernel execution.}\end{aligned}$$

The certificate direction proves progressively more uniform statements without changing that execution:

$$\begin{aligned}\text{path bound} &\longrightarrow \text{input-dependent exact budget}\\ &\longrightarrow \text{uniform measured runtime} \longrightarrow \text{polynomial runtime.}\end{aligned}$$

This distinction explains why asymptotics joins only at the complexity layer. The definitions `IsPolyBounded` and `IsNegligible` do not participate in exact

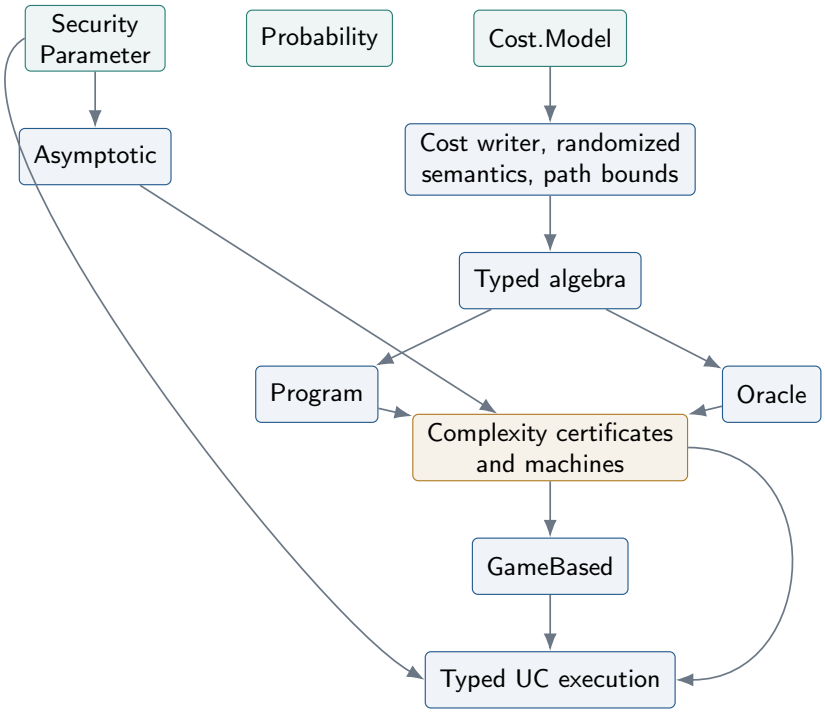

Fig. 1. Permitted logical flow. Arrows denote admissible dependency directions, not necessarily direct Lean imports. Probability, security parameters, and the base cost model are independent roots.

evaluation. Negligibility is stated using the eventual bound on $|f(n)|$, preventing a negative function from becoming negligible merely because it lies below a positive threshold.

2.2 Stable semantic boundaries

Cost parameters do not enter ordinary games, advantages, or correctness statements. A security predicate instead selects an adversary cost model and natural-number measure and then quantifies inhabitants of the resulting PPT-machine interface whose exact run and claimed runtime also carry an independent operational-admission witness. Polynomial path annotations do not construct that witness. This preserves the distinction between the cost model used to describe a construction and the independently selected operational model used to admit adversaries. Similarly, real and ideal UC worlds share their environment, routing policy, corruption policy, and fuel convention; only the installed protocol/adversary versus functionality/simulator components differ.

Table 1. Authoritative objects by layer. A higher row may export evidence consumed below it, but does not replace the semantic object it certifies.

Layer	Authoritative object	Evidence or public view
Cost	joint value–cost computation	support-wise path bound; cost erasure
Algebra	typed exact operation handler	erased operation law; operation bound
Program	one typed program and interpreter	execution relation; structural bound
Oracle	one exact recursive interpreter	trace/count projections; composition bound
Complexity	one dependent exact machine run	exact, measured, and polynomial certificates
GameBased	cost-erased experiment	advantage, hardness, indistinguishability
UC	typed closed-world kernel run	fuel/cost certificates; real/ideal decision distributions

The same pattern appears repeatedly: packaging may change as exact semantics is exposed, but cost-erased probability statements remain the public boundary.

3 Exact Resources, Typed Algebras, and Programs

This section describes the semantic spine used by finite cryptographic computations. Resource annotations are first-class results of execution, not metadata reconstructed from an algorithm after the fact. A typed primitive handler is the only source of primitive costs, and a program obtains both its ordinary distribution and its resource certificates from that same handler. Figure 2 summarizes the resulting separation.

3.1 Ordered sequential resource semantics

The structure `CostModel` packages a type C, an additive monoid $(C, 0, +)$, a partial order $\leq$, and monotonicity of addition in both arguments. Exact sequential execution uses $+$ in evaluation order. No commutativity requirement is imposed, so a model may retain order-sensitive information rather than merely count scalar steps. The order and its two monotonicity laws enter only when exact paths are compared with budgets.

The optional capability `WorstCaseCostModel` adds an operation $\texttt{sup} : C \times C \to C$ and proves its least-upper-bound laws against the very same order stored by the base model. It deliberately does not carry a second order instance. The concrete definitions `CostModel.nat` and `WorstCaseCostModel.nat` instantiate

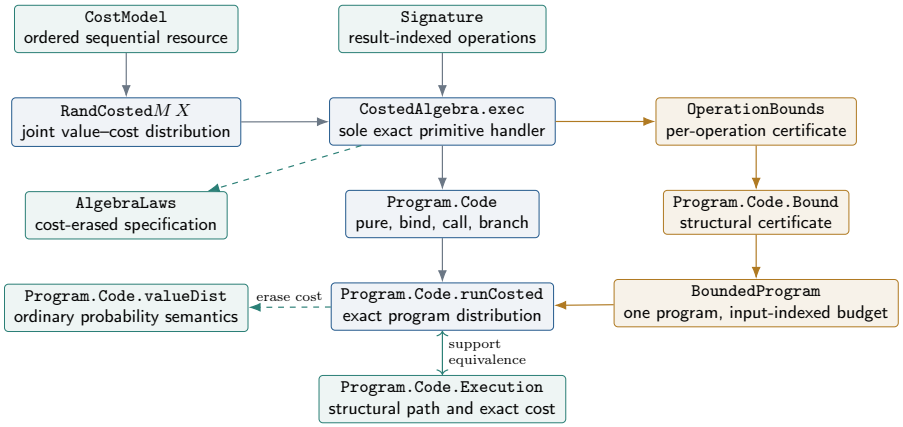

Fig. 2. The exact semantic spine and its proof-only side layers. Blue arrows construct execution; dashed arrows forget costs; gold arrows build upper-bound evidence. `AlgebraLaws` and `OperationBounds` do not define a second interpreter. The two-headed edge is the proved correspondence between structural execution and the support of `runCosted`; it does not identify the execution relation with `valueDist`.

sequential composition with natural addition and worst-case choice with max, respectively; they are examples, not assumptions of the generic semantics.

For a value type X, `Costed`$M\,X$ is a pair $\langle x, c\rangle$ with $x : X$ and $c : C$. Its writer bind is

$$\langle x, c_1\rangle \gg\!= f = \langle y, c_1 + c_2\rangle \quad \text{when} \quad f(x) = \langle y, c_2\rangle.$$

The order of $c_1 + c_2$ is therefore observable for noncommutative models. Theorems `Costed.pure_bind`, `Costed.bind_pure`, and `Costed.bind_assoc` establish the monad laws from the additive-monoid laws; `Costed.bind_cost` makes the single accumulation point explicit.

Randomness is added without marginalizing cost:

$$\mathsf{RandCosted}_M\, X \;:=\; \mathsf{PMF}(\mathsf{Costed}_M\, X).$$

Thus a sampled value and the resource consumed on the path that produced it remain correlated. `RandCosted.bind` combines PMF bind with writer bind, and its lawful-monad proof includes associativity in the model's execution order. `RandCosted.valueDist` is only the image under the value projection. In particular, the erasure theorem for bind is

$$\mathsf{valueDist}(d \gg\!= k) = \mathsf{valueDist}(d) \gg\!= (x \mapsto \mathsf{valueDist}(k(x))),$$

Within namespace `RandCosted`, this is theorem `valueDist_bind`. The construction `sampleWithCost` attaches a possibly value-dependent cost to each sample; its value-erasure theorem proves that this annotation leaves the sampled distribution unchanged.

The sole generic path-bound predicate is

$$\texttt{RandCosted.CostBound}(d, b) \;\equiv\; \forall r \in \mathrm{supp}(d),\; r.\mathsf{cost} \le b. \tag{1}$$

Its `pure`, `map`, `bind`, and `weaken` theorems are the basic proof algebra reused by later certificate layers. In particular, `RandCosted.CostBound.bind` concludes a budget $b_1 + b_2$, in that order, from support-wise bounds for the first computation and every continuation.

Projection is observational, not semantic. The structure `NatMeasure`M is a monotone additive homomorphism $\mu : C \to \mathbb{N}$. It marks the boundary at which a rich exact resource is observed as conventional runtime. Theorems `NatMeasure.map_add` and `NatMeasure.map_nsmul` cover sequential composition and finite repetition. The map `RandCosted.mapCost` acts on every exact path and is a writer-monad morphism; the value-erasure theorem `RandCosted.valueDist_mapCost` proves

$$\mathsf{valueDist}(\texttt{mapCost}\ \mu\ d) = \mathsf{valueDist}(d).$$

Monotonicity transports Eq. (1) to the projected natural-number budget via `RandCosted.CostBound.mapCost`. This projection therefore supports complexity proofs while preserving both the underlying value distribution and the unprojected exact run as the authoritative object.

3.2 A typed algebra with separate laws and bounds

An operation family often contains primitives with different result types: sampling may return a scalar, scalar action returns a carrier element, and addition returns another carrier element. `Signature` represents this heterogeneity directly with a result-indexed family

$$\texttt{Signature.Op} : (R : \mathsf{Type}) \to \mathsf{Type}.$$

Consequently an inhabitant of `S.Op`$\,R$ carries, in its type, the fact that handling it returns R. `Signature.sum` forms the disjoint union of two capabilities pointwise in the result index. Its companion `CostedAlgebra.sum` dispatches to the corresponding exact handler, so algorithms can request only the primitives they use rather than populate a monolithic record with dummy fields.

The handler itself is intentionally small:

$$\texttt{CostedAlgebra.exec} : \texttt{S.Op}\ R \to \mathsf{RandCosted}_M\ R.$$

It is the single source of both operation outcomes and their exact path costs. Two structures describe independent facts about that handler. First, `AlgebraLaws`A stores a cost-erased PMF specification and proves that erasing `A.exec` yields it. Second, `OperationBounds`A chooses a budget for each operation and proves Eq. (1) for every supported handler result. Neither structure executes an operation or stores a competing exact cost. Their respective `sum` constructors preserve this separation under signature composition.

Table 2. Algebra and program objects, with the information each is allowed to own.

Object	Owns	Does not own
`Signature`	typed operation family	PMFs, costs, or bounds
`CostedAlgebra`	exact joint operation handler	mathematical law or second cost table
`AlgebraLaws`	erased PMF specification and equality	execution semantics
`OperationBounds`	independently chosen operation budgets	exact operation costs
`Program`	one input-indexed typed body	a budget or duplicate bounded body
`BoundedProgram`	that program and a structural certificate	a second algorithm or interpreter

The library supplies typed operation families for addition, negation, subtraction, scalar action, multiplication, and sampling. For example, within namespace `SampleOperation`, `algebra` accepts an arbitrary exact joint value–cost distribution. Its `laws` constructor separately identifies the erased PMF, while `bounds` separately certifies a path budget. Deterministic arithmetic handlers follow the same pattern: their exact cost functions occur only in the corresponding `algebra` constructor, their `laws` constructors erase to the relevant Mathlib operation, and their `bounds` constructors compare exact costs with caller-selected budgets.

Design principle. Changing a mathematical specification cannot change execution, and changing an upper-bound certificate cannot change either values or exact costs. To change an exact primitive cost, one must change the handler whose execution is already used by every program consumer.

3.3 One program, one interpreter, and support-complete executions

For a fixed algebra A, `Program.Code` $A\,R$ has four constructors: `pure` returns a value at zero cost; `bind` sequences higher-order code; `call` invokes a typed primitive; and `branch` selects between two code values of the same result type using a Boolean condition. External input is not baked into this syntax. Instead,

$$\mathsf{Program}\ A\ \mathit{Input}\ \mathit{Output} \quad \text{stores} \quad \texttt{body} : \mathit{Input} \to \texttt{Program.Code}\ A\ \mathit{Output}.$$

This boundary permits the exact resource budget to depend on the supplied input while keeping the program code structurally inductive. Since `bind` is higher order, a primitive result may determine all subsequent operations and the Boolean supplied to `branch`.

The recursive definition `Program.Code.runCosted` is the only evaluator. It maps `pure` to zero-cost writer pure, `bind` to randomized writer bind, `call` to `A.exec`, and `branch` to the selected recursive run. `Program.Code.valueDist` is

definitionally the value projection of this result. Its compositional lemmas cover all four constructors, and `Program.Code.valueDist_call_eq` connects a call to its `AlgebraLaws` specification without adding an alternative evaluator.

The inductive relation `Program.Code.Execution`*code*, *value*, *cost* records the actual structural path. Its bind constructor adds the first and continuation costs in order; its call constructor requires membership in the support of `A.exec`; and its two branch constructors record only the selected branch. Within namespace `Program.Code`, the two directions are proved separately and combined by `execution_iff_mem_support_runCosted`:

$$\texttt{Execution}\ p\ x\ c \quad \Longleftrightarrow \quad \langle x, c\rangle \in \mathrm{supp}(\mathsf{runCosted}(p)). \tag{2}$$

Equation (2) is stronger than a one-way soundness statement: every semantic support point has a structural witness, and every structural execution occurs in the exact interpreter's support. It also prevents a proof-only execution relation from silently describing paths absent from the probabilistic semantics.

3.4 A minimal first-order operational subset

The namespace `Computation.FirstOrder` provides a smaller syntax for code that must cross the operational PPT boundary without an external code-validity assumption. Its type codes contain named base carriers, unit, Booleans, and products. Typed de Bruijn variables index an explicit heterogeneous `Env`, and pure `Expr` syntax contains variables, constants, pairs, and projections. In particular, there is no function type.

A first-order signature indexes every primitive by both its argument type and result type. A `Code.call` therefore stores an operation label and an argument expression; its continuation is code under one additional typed variable, rather than a Lean function. Together with represented return, pure-let, and branch nodes, this gives a typed straight-line language whose sole evaluator again produces `RandCosted`. Ordinary semantics is only cost erasure of that evaluator.

The built-in primitive layer contains addition, negation, subtraction, scalar action, multiplication, and finite uniform sampling, with disjoint-sum composition. Arithmetic primitives invoke the corresponding bottom algebra operation at a declared exact cost. The sampler is not supplied as an arbitrary PMF: its handler is fixed to `uniformPMF` on a finite nonempty carrier, and its erasure theorem recovers that distribution. Structural `ValidAlgebra` evidence records that an exact handler is assembled only from these primitives. A `FirstOrderOperationalCode` then packages one program, such algebra evidence, an exact path bound, and a measured uniform runtime. Section 4 explains how this package yields operational admission.

3.5 Structural and input-dependent bounds

At code level, `Program.Code.CostBound`*code*, *budget* is only an abbreviation for Eq. (1) applied to `runCosted`(*code*). The structural certificate is indexed by the existing code:

`Program.Code.Bound`*bounds*, *code*, *budget* does not store the code as another field. Its constructors mirror the semantic evaluator:

- `Bound.pure` certifies budget 0, and `Bound.call` consumes the independently supplied `OperationBounds`;
- `Bound.bind` combines budgets as $b_1 + b_2$ in execution order, requiring one common continuation budget valid for every intermediate value;
- `Bound.branchOfBounds` accepts any caller-proved common upper bound for the two branch budgets; and
- `Bound.branchSup` derives that common budget automatically only when the model provides `WorstCaseCostModel`.

The distinction between the last two constructors is material. Exact interpretation needs no least-upper-bound operation. Models without such an operation remain usable, provided a caller supplies both inequalities to an explicit common budget. `Bound.weaken` then permits a certified budget to be widened using the model's partial order.

At the outer boundary,

$$\texttt{Program.CostBound}\ p\ B\ \equiv\ \forall i,\ \texttt{Program.Code.CostBound}\ (p.\texttt{body}\ i)\ (B(i)),$$

so budgets may depend on input. `BoundedProgram`*bounds*, *budget* stores one `Program` plus, for each input, a structural `Code.Bound` for that same body. The theorem `BoundedProgram.costBound` forgets the structure to obtain the semantic path bound. Its companion support corollary applies that bound directly to an exact supported result. Thus bounded and unbounded views do not duplicate an algorithm: the former is the latter paired with evidence about the exact interpreter already used for probability semantics.

Scope boundary. The general `Program` syntax charges only calls handled by the algebra's `exec` field. Lean-level construction of its higher-order continuations and evaluation of the Boolean already supplied to `branch` are not assigned an implicit host runtime. The first-order subset removes those continuations, but is deliberately straight-line: recursion, RAM state, bit-level representations, and adequacy of bottom algebra costs for a concrete platform remain outside this semantic layer.

4 Machines, Certificates, and Oracle Composition

4.1 One dependent machine core

Machine input and output types may depend on the security parameter, and the output may additionally depend on the concrete input. The common computation shape is

$$\begin{aligned}&\mathsf{RandomizedComputation}\ M\ I\ O\\ &\quad = (s : \mathsf{SecPar}) \to (x : I(s)) \to \mathsf{RandCosted}\ M\ (O(s, x)).\end{aligned}$$

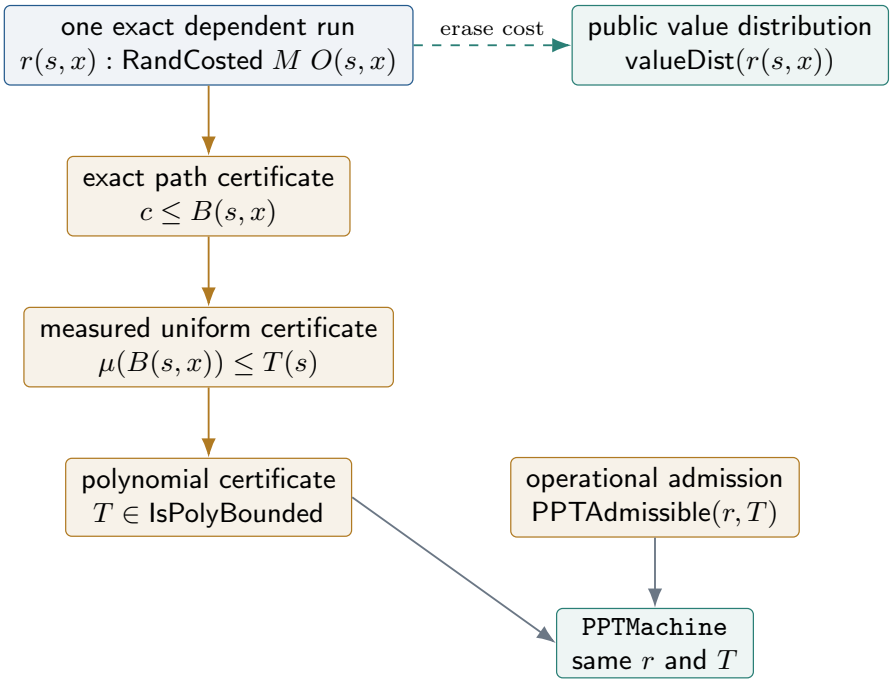

Fig. 3. Certificate refinement never rewrites the exact run. Polynomial annotation and operational admission must both be present at the PPT boundary; canonical first-order code derives the latter internally, while probability erasure is orthogonal.

The structure `ProbabilisticMachine` stores exactly one such `run`. Ordinary I/O is represented by constant families, while search problems use the dependent output directly. Thus there is no separate ordinary/dependent machine hierarchy and no duplicated algorithm when a certificate is attached.

The exact annotation chain and the separate PPT-admission boundary are shown in Figure 3. `ExactCostCertificate` supplies an input-dependent budget $B(s,x)$ and proves `RandCosted.CostBound` for the existing run. Given a `NatMeasure` μ, `RuntimeCertificate` additionally supplies an input-independent runtime $T(s)$ and proves $\mu(B(s,x)) \le T(s)$. Finally, `PolyRuntimeCertificate` proves `IsPolyBounded` T. These statements certify the annotated cost model, not Lean host evaluation. The structure `PPTMachine` therefore also stores `PPTAdmissible` for the same exact run and the same claimed runtime. This admission is a transparent `OperationalRealization`: it exposes a model, validated code object, denotation equation, and claim equation. No theorem derives its `OperationalModel.ValidCode` witness from the annotation chain alone. For `FirstOrderOperationalCode`, however, the reified syntax and structural algebra witness provide the missing operational evidence internally.

`RuntimeCertificate.measuredCost_le_runtime` composes path soundness, monotonicity of μ, and the uniform bound to show that every supported

exact result measures below $T(s)$. Value-only dependent maps preserve all three annotation-level certificate layers, but they do not automatically preserve operational admission. Likewise, `TimedMachine.ofBoundedProgram` and `PPTMachine.ofBoundedProgram` install the exact run of a bounded typed program; the PPT constructor requires a separate admission witness because the current program syntax retains higher-order host boundaries. The corresponding `valueDist_run_ofBoundedProgram` lemmas show that the adapter changes no value distribution. By contrast, `PPTMachine.ofFirstOrderCode` consumes a represented straight-line program, a structurally validated bottom algebra, its exact path certificate, and a measured runtime. It constructs both the timed machine and its canonical operational realization without an external `ValidCode` hypothesis. The resulting machine family runs the same reified code at every security-parameter index; the index cannot select a hidden Lean continuation.

The game-based layer consumes this interface. Hardness and indistinguishability predicates select an adversary cost model and measure and quantify all inhabitants of the selected `PPTMachine` type. This is more precise than saying "all computable adversaries": every inhabitant includes an operational-admission witness indexed by its exact semantics and runtime. The generic library deliberately supplies no witness for an arbitrary Lean function. Such code must either be expressed in the first-order subset or use an explicitly audited external model through `ExternalValidCode`.

4.2 Typed oracle syntax and one interpreter

An `OracleSpec` is a result-indexed family of query and response types. Oracle program syntax contains `pure`, `bind`, `liftCosted`, and `query`; a query node contains its name and typed payload but no local cost. Issuing a query is a result-indexed `QueryIssue` operation. Its exact caller-side cost comes only from a `CostedAlgebra` handler, while an independent operation bound can certify it.

`CostedOracleEnv` handles implemented-oracle work. `OracleEnv.zeroCost` lifts an ordinary `OracleEnv` into that exact interface. `CostedOracleEnv.erase` returns the ordinary environment by erasing implementation costs. Consequently there is no second ordinary implementation to keep synchronized.

The sole structural recursion is `Oracle.Program.runExact`. Its result `ExactRunResult` records six logically distinct fields:

$$(\mathit{value}, \mathit{state}, \mathit{trace}, c_{\mathrm{local}}, c_{\mathrm{oracle}}, c_{\mathrm{total}}).$$

The total cost preserves chronological issue/environment order. The local and oracle fields are audit projections; in a noncommutative model they are not definitionally regrouped into the total. All other views are maps of this distribution: `runCosted` retains value and total cost, `runWithEnv` erases costs and final state, and `runTrace` retains value and trace. The principal erasure law against the same implemented environment is

`valueDist_runCosted_eq_runWithEnv_erase`.

For environment-independent structural reasoning, `PossibleExecution` records a possible value, local cost, and trace. The proved direction is

$$r \in \mathrm{support}(\mathsf{runExact}(p, s, E, \sigma)) \implies \mathsf{PossibleExecution}(p, r.value, r.c_{\mathrm{local}}, r.trace).$$

There is intentionally no converse: the structural relation overapproximates responses and need not describe a path supported by a particular environment.

4.3 A closed exact-to-polynomial composition theorem

Caller and implementation certificates are separated along the boundary where they can be reused. A `TimedOracleMachine` certifies an input-dependent local budget, per-name query budgets, an independent total-query budget, a uniform local runtime, and a uniform total-query runtime. An `OracleImplementation` stores the exact environment; `TimedOracleImplementation` adds an input-dependent per-query budget, a uniform measured query runtime, and monotonicity of repeated addition; and `PPTOracleImplementation` proves polynomiality.

For a caller budget B_{local}, total-query budget Q, and implementation budget B_E, the exact composed budget is

$$B_{\mathrm{exact}}(s, x) = B_{\mathrm{local}}(s, x) + \mathsf{repeatCost}_M(Q(s, x), B_E(s, x)). \tag{1}$$

Here `repeatCost` is repeated sequential addition in the model's `AddMonoid`; it is $Q \cdot B_E$ only for the natural-number instance. The corresponding uniform runtime is

$$T(s) = T_{\mathrm{local}}(s) + T_{\mathrm{queries}}(s)\, T_E(s). \tag{2}$$

The proof first bounds exact chronological cost by separated local and oracle projections using `CostExchange`. It then bounds oracle work by query count and repeated implementation budget, and uses the caller's structural total-query certificate. `NatMeasure.map_nsmul` turns repeated exact cost into multiplication in $\mathbb{N}$, yielding Equation (2). `TimedOracleMachine.compose` constructs the timed machine with this exact budget and runtime; `PPTOracleMachine.compose` consumes both the polynomial closure proof and an independent admission witness for that closed run and runtime. Finally, `PPTOracleMachine.compose_runDist` shows that the composed PPT machine has exactly the cost-erased semantics of the authoritative environment. Query counts are never inferred from runtime or from a unit query charge.

5 A Typed Universal-Composability Execution Layer

The UC subsystem instantiates the same exact-semantics discipline at the level of interacting machines. Its operational core is a typed, single-activation FIFO kernel. Protocols, ideal functionalities, environments, adversaries, simulators, and networks are role-specific wrappers around exact interactive machines; their public Boolean games are obtained only after running the kernel and erasing its costs.

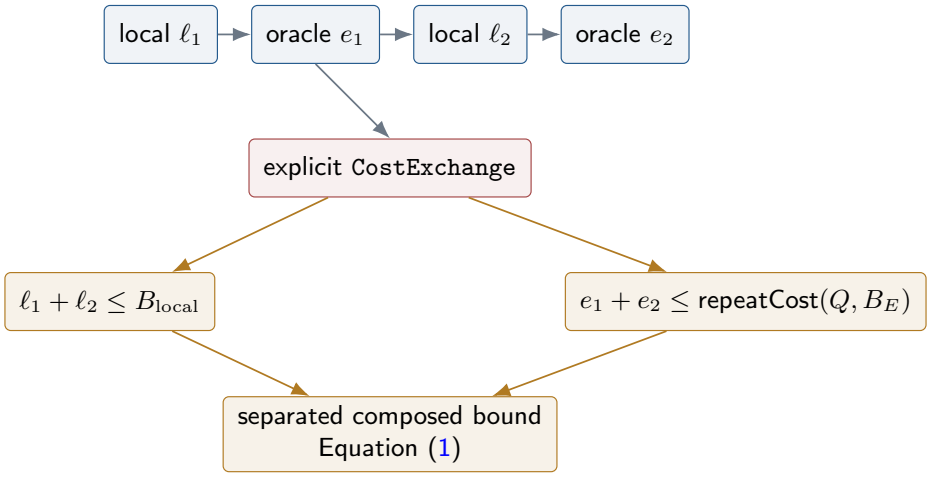

Fig. 4. The interpreter keeps chronological, possibly noncommutative cost. Only the coarse theorem regroups caller and oracle work, and therefore asks explicitly for `CostExchange`.

Design principle. The global scheduler owns routing, queue order, corruption, and kernel bookkeeping. An ITM owns only its address-indexed local state and its exact handlers. Neither side can manufacture a message whose payload, endpoints, and connection capability disagree.

5.1 Sessions, ports, and address-owned machines

A session identifier `SID` consists of a root tag and a list-valued child path. The operation `SID.child` appends one tag, while `SID.Ancestor` is prefix inclusion restricted to an equal root. A machine address is the pair of a session identifier and a machine name. This representation makes independent top-level executions disjoint and retains the complete nested-session path in every address.

The structure `PortSchema` gives, for each address, direction, and payload type, a type of ports. A value of `CanConnect` witnesses a permitted connection between an output and an input port carrying the same payload type. It also determines a `RoutingPolicy`: direct delivery or one of the observable adversarial routes, with authenticated, forgeable, and broadcast capabilities distinguished. A separate `CanSendAs` capability authorizes a controller to claim a source address. Accordingly, `Message`, `Incoming`, and `Emission` carry their payload type, typed endpoints, and connection witness. Ill-typed delivery is therefore excluded when a message is constructed rather than detected later by the interpreter.

An `ITMFamily` is indexed by the security parameter and address. It owns dependent families `State`, `Leakage`, `Erasure`, and `Output`, together with exact `init`, `activate`, `applyErasure`, and `leak` handlers. An activation consumes

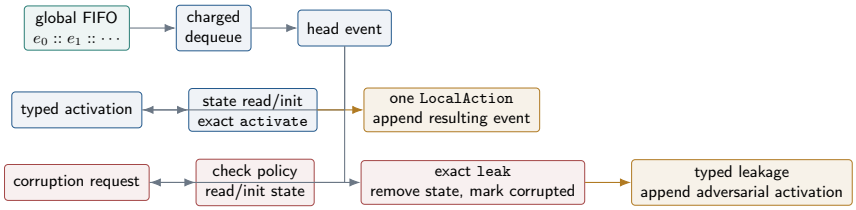

Fig. 5. The authoritative scheduler consumes one FIFO event per kernel step. Corruption is an independently queued transition, so leakage and state removal cannot be hidden inside the requesting activation. Rejected requests leave the corrupted set unchanged.

either a typed incoming message or `resume`. It returns a new local state and exactly one `LocalAction`: yield, emit, authorized send-as, erase, spawn, request corruption, or output. A computation that needs several emissions must store a continuation and request later resume activations. In particular, broadcast permission never triggers hidden kernel fanout; a broadcast component serializes its recipients as ordinary activations.

5.2 A single FIFO and an explicit corruption transition

The global `Configuration` uses a dependent function as its local-state store: the address determines the type of the optional stored value. No `Any` or dynamic cast occurs. The configuration also stores a list of `QueuedEvent`, the corrupted-address set, an optional terminal output, and a typed audit trace. Both ordinary activations and corruption requests occupy the same FIFO; enqueue appends at the tail and `Kernel.stepOne` removes only the head.

The corruption policy is deliberately separate from erasure semantics. `CorruptionPolicy` specifies the admissible corrupted sets and the legal one-address transition. Its invariant inside a configuration states both that the current set is admissible and that a corrupted address retains no honest state. The supplied `incorruptible`, `static`, and `dynamic` policies are restrictions of this same transition interface.

Figure 5 shows the two event paths. An honest activation is lazily initialized, activated once, and allowed one action. An activation addressed to a corrupted target is redirected through the network-control adapter. A permitted corruption request reads the current state (initializing a dormant machine if necessary), calls the exact leakage handler, removes the honest state, inserts the target into the corrupted set, and enqueues a typed leakage notification. Subsequent send-as succeeds only when both the static `CanSendAs` witness exists and the claimed source belongs to this exact configuration's corrupted set. A rejected attempt is charged and audited but cannot enqueue its forged emission.

Erasure follows the same explicit chronology. Processing an `erase` action calls `applyErasure`, writes the resulting state, records the request, and queues a resume. A later corruption therefore leaks that stored post-erasure state. The kernel proves the ordering and state-removal invariant; the meaning and adequacy

of a particular erasure or leakage function remain obligations of the installed ITM.

Kernel bookkeeping is itself costed. `KernelOperation` is a typed signature covering dequeue, initialization, state access, routing, enqueue, erasure, corruption, and termination, and `KernelAlgebra` is its unique exact handler. The named `KernelAlgebra.zero` handler is available when an explicitly free bookkeeping model is intended; zero cost is not silently built into the runner.

5.3 Finite execution and the closed-world certificate chain

The sole finite runner is

$$\texttt{Kernel.runCosted} : \mathbb{N} \to \mathsf{Configuration} \to \mathsf{RandCosted}\ M\ \mathsf{ExecutionResult}.$$

Its fuel counts scheduler steps only. An `ExecutionOutcome` is a typed machine output, timeout, or deadlock; the final configuration and trace remain in `ExecutionResult`. Fuel is not a checked cost budget. At the public Boolean boundary, `ExecutionOutcome.toBool` maps timeout and deadlock to false, and `Kernel.decisionDist` first erases the exact costs. Thus a failed certificate cannot change a branch or probability: certificates are propositions about the runner, not inputs inspected by it.

The complexity proof is built in three stages. First, `ComponentCostCertificate` bounds initialization, activation, erasure, and leakage handlers, while independent `OperationBounds` bound the kernel algebra. A `StepCostCertificate` places all of these charges below one security-parameter-dependent atom. The current branch audit fixes a conservative maximum of ten atomic charges; `step_sound` follows the actual `stepOne` branches and pads shorter branches using nonnegativity. Then `runCosted_sound` proves the ordered repeated-step budget

$$c_{\mathrm{run}} \ \le\ \mathsf{repeatActivationCost}_M(f, c_{\mathrm{step}}).$$

Because the resource monoid need not be commutative, repetition and bind keep the scheduler's left-to-right order.

Second, a `FuelCertificate` supplies semantic evidence that the selected activation limit has no timeout in exact support and that adding fuel leaves the erased value distribution unchanged. This is a genuine termination and stability obligation; it is not inferred from a resource upper bound. Third, `PPTExecutionCertificate` combines the step certificate, activation limit, a natural-number bound on the measured step cost, polynomial proofs for both functions, an independent operational admission of the exact closed runner at the claimed runtime, and the fuel certificate. Its exported runtime is

$$T(s) = f(s)\, T_{\mathrm{step}}(s),$$

and `NatMeasure.map_nsmul` connects the exact repeated cost to this product. This annotation proof does not generate operational admission; the stored witness is reused by `toPPTMachine`. The resulting `PPTMachine` executes the original `Kernel.runCosted`; it does not execute a projected or reconstructed runner.

For comparing two worlds, `CertifiedWorldPair.commonFuel` is the pointwise maximum of their activation limits. Polynomial closure of maximum, the two extended fuel certificates, and the theorems `real_noTimeout`, `ideal_noTimeout`, `real_stable`, and `ideal_stable` establish a shared finite experiment. In particular, changing the simulator cannot change the real game merely by changing the selected common fuel, and changing the adversary cannot change the ideal game for that reason; these independence properties are proved from stability.

5.4 Real and ideal worlds

The global `ClosedWorldAddress` is a disjoint sum of environment, system, adversarial, and network addresses. The function `dispatchFamily` selects the corresponding exact handler definitionally and preserves the address-indexed local-state type. `composeReal` assembles environment–protocol–adversary–network, whereas `composeIdeal` assembles the same role-separated interface with environment–functionality–simulator–network; see Figure 6. `Protocol` and `IdealFunctionality`, and likewise `Adversary` and `Simulator`, are distinct structures even though each ultimately owns an `AddressedITM`.

An `Environment` must expose an address-indexed `ULift Bool` output. The closed-world decision function accepts that output only when its source is in the environment summand. Outputs from the protocol, functionality, adversarial role, or network are mapped to false rather than being mistaken for the distinguisher's bit. A real/ideal certified pair additionally shares the environment and corruption policy, and an `Experiment` fixes the same port schema, network, protocol, and ideal functionality used by all quantified roles.

Computational UC emulation is then stated with the standard dependency order:

$$\forall\, \mathcal{A} \in \mathsf{PPT},\quad \exists\, \mathcal{S} \in \mathsf{PPT},\quad \forall\, \mathcal{Z} \in \mathsf{PPT},\quad \mathsf{Real}_{\Pi,\mathcal{A},\mathcal{Z}} \;\approx\; \mathsf{Ideal}_{\mathcal{F},\mathcal{S},\mathcal{Z}}. \tag{3}$$

Here each quantified role carries a `PPTAddressedITMCertificate` for the experiment's explicit cost model (M) and `NatMeasure`; the complete role certificate includes operational admission of the exact handlers and claimed runtime. Complete closed-world builders additionally provide an execution certificate, with its own admission for the composed runner, that accounts for the protocol/functionality and network. The relation `Indistinguishable` compares the two cost-erased Boolean games. `PerfectUCEmulates` replaces indistinguishability by pointwise equality, and `PerfectUCEmulates.ucEmulates` derives the computational statement by showing that the advantage is identically zero.

5.5 Operational contexts and composition

Universal composition is phrased over `ExecutableExperiment`, whose real and ideal worlds are structurally assembled from exact components and their certificates. A `SystemHole` is one transformation of a system-owned `AddressedITM`; the same `fill` produces both the outer protocol and outer functionality.

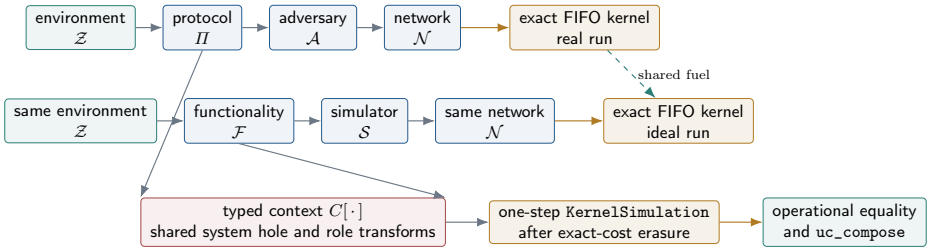

Fig. 6. Real and ideal worlds are concrete four-component executions. A context transforms both system roles through one typed hole, while its operational premise is a commuting single-step kernel simulation.

`ContextBuilder` also transforms the shared policy and network and must construct the corresponding real and ideal execution data. Hence the outer experiment cannot be an unrelated preassembled game attached to a nominal hole.

Renaming is similarly explicit. `WorldRenaming` consists of four role-preserving injective address maps, and `PortTransport` maps activations, emissions, and send-as capabilities while preserving targets and routing policy. A `KernelSimulation` then maps dependent states, leakage, erasures, outputs, configurations, queues, corrupted sets, and traces. Its central premise is a commuting `Kernel.stepOne` square after exact cost erasure, together with initial-configuration, decision, policy, and network-adapter laws. Induction yields finite-run and Boolean-decision commutation.

A `Context` additionally provides PPT-preserving transformations `contextAdversary` and `contextEnvironment`, plus an environment-independent `plugSimulator`. Its real and ideal premises are simulations between the actual certified worlds produced by the builder and the transformed inner roles. Identity and sequential composition compose these single-step simulations; plugging is definitionally unital and associative for structurally executable experiments. Theorems `real_operational` and `ideal_operational` turn the simulations into equalities of the compared Boolean games. Finally, `uc_compose` applies the inner simulator to the context-transformed adversary and lifts it with `plugSimulator`, preserving the quantifier order in Equation (3).

Scope boundary. The current context interface is intentionally endomorphic: it keeps one global port schema and the same four local address types, up to role-preserving injective endomaps. Its simulation theorem preserves erased operational behavior, not exact path costs. The context must construct new PPT-certified executions; polynomial closure is enforced by these output types rather than inferred from an arbitrary host-language function.

5.6 The layered-MPC bridge

The `Layered` namespace connects the generic kernel to a concrete family shape without postulating a separate execution semantics. Parameters record the number of parties per layer, the number of layers, and a per-cell corruption threshold. A party identifier contains both layer and position, and its system address additionally contains the full `SID`. The predicate `Corruption.Eligible` permits only party addresses and enforces the threshold independently for every exact session–layer pair. Thus a child session or a different root cannot consume another cell's corruption budget.

A `PartyStep` is an actual exact ITM, not a detached transition relation. `SystemComponents` installs party machines, an explicit broadcast manager, an explicit corruption manager, and typed boundary machines; its total address dispatcher becomes the real `Protocol`. Likewise, `MPCFunctionality` contains an actual addressed ITM and becomes an `IdealFunctionality`. `ExecutableLayered` supplies the shared layered corruption policy, both initial configurations, and both `PPTExecutionCertificate`s, and `toExecutableExperiment` places these components directly into the generic real/ideal and context infrastructure.

Scope boundary. This bridge establishes executable typed wiring and certificate obligations; it is not by itself a security proof for a particular layered MPC protocol. Concrete party algorithms, broadcast behavior, ideal functionality behavior, simulator construction, and the simulations required by `uc_compose` must still be supplied. Likewise, the finite runner models activation-bounded executions; liveness beyond a provided `FuelCertificate` is outside the present kernel theorem.

6 Cryptographic Case Studies

The abstractions are exercised by four construction families. The point of these examples is not the novelty of their reductions, but the fact that exact execution, cost erasure, and complexity evidence use the same interfaces. Table 3 summarizes the mechanized boundary.

Table 3. Construction-specific evidence connected to the common infrastructure.

Family	Exact program and bound evidence	Cost-erased result
OTP	setup, key generation, encryption, and decryption programs; parameter and family efficiency certificates	correctness and perfect one-time security
DLog	setup, challenge, and sampling programs; parameter/family efficiency certificates	search problem and all-PPT assumption
DDH	setup, real/random challenge and sample programs over a shared decisional cyclic action	distinguishing problem and all-PPT assumption
ElGamal	key generation, encryption, and decryption programs; exact encryption-path theorem; timed adapter	scheme PMFs and correctness

One-time pad. The OTP parameter contains only the finite additive group and operations that the construction uses. There is no dummy scalar type or blanket scalar-action instance. `keygenProgram`, `encryptProgram`, and `decryptProgram` call typed sampling, addition, and subtraction operations; their bounded variants attach certificates to those same programs. Theorems such as `scheme_encryptDist` expose the erased distribution, and `perfectOneTimeSecure` proves exact equality of the left and right experiments before deriving `oneTimeSecure` for every selected PPT adversary.

DLog and DDH. `CyclicAction` supplies a finite additive carrier, a finite scalar type, an action, a generator, and generator surjectivity. The stronger DDH parameter `DecisionalCyclicAction` adds a commutative scalar monoid and the action laws needed by DDH. Its `MulAction` is derived from the inherited action, avoiding a second incoherent instance. Setup produces the dependent public parameter, after which result-indexed operations sample values of `pp.Scalar` and act on `pp.Carrier`. Program erasure lemmas connect these exact paths to `dLogProblem` and `ddhProblem`. Their `Assumption` definitions still quantify every PPT machine admitted by the explicit adversary model and measure.

ElGamal. ElGamal reuses the DDH public parameter and handler rather than copying its algebraic laws. Encryption performs exactly one scalar sample, two scalar actions, and one addition. The theorem `encryptProgram_exactCost` decomposes every supported result into those four supported primitive results and proves that the recorded cost is their ordered sum. This theorem consults the exact handler, not an upper-bound certificate. Independently, `encryptProgram_costBound` certifies the budget and `encryptTimedMachine_runDist` proves

that the machine adapter preserves the encryption PMF. The existing `correct` theorem is stated over the cost-erased scheme distributions.

The current library does not yet prove ElGamal IND-CPA security from the DDH assumption. The syntax and generic IND-CPA boundary exist, but presenting the reduction as completed would overstate the mechanized result.

7 Mechanized Validation

This section reports a repository snapshot, not a runtime-performance benchmark. On 2026-08-05 the migrated tree contained 13,994 lines across 115 production Lean files and 3,911 lines across 14 Lean regression modules. Infrastructure itself comprised 75 modules. A source scan found 194 compile-time `example`, `theorem`, or `lemma` declarations in `CryptoTest`.

The acceptance sequence checked the import DAG and then completed the library target (1,798 jobs), construction target (1,777 jobs), theorem-regression target (3,240 jobs), and default target (3,269 jobs), without warnings. Static audits found no `sorry`, `admit`, explicit `axiom`, or `unsafe` in production or test sources, and `git diff -check` reported no whitespace errors. The three admission interfaces are transparent specializations of one structured operational-realization relation. `OperationalModel.ValidCode` has internal constructors for the canonical first-order interpreter, while `ExternalValidCode` is the sole opaque trust-boundary relation for other backends. The repository provides no global validation fact or generic constructor for host functions. These counts describe the current local snapshot; they are expected to change as the library evolves.

The CI workflow runs the import checker before Lean and explicitly names the `Crypto`, `CryptoTest`, and default targets on GitHub Linux. It also generates API documentation separately from the source `docs/` tree. This guards the logical hierarchy and the build boundary, while the theorem regressions guard semantic equalities and exact cost order.

7.1 What the validation establishes

The tests demonstrate that the interfaces are not merely specialized natural-number wrappers: the noncommutative and multidimensional models elaborate and their laws compose through the generic APIs. They also exercise the exact chronology that would be hidden by a commutative model, including query issue cost followed by environment cost and FIFO kernel actions. Finally, existing cryptographic results compile against the cost-erased views, providing a regression check that the complexity refactor did not silently alter their probability semantics.

The builds do not establish adequacy with physical execution time, completeness of any overapproximating path relation, or security of an uninstantiated UC protocol. Those boundaries are the subject of the next section.

Table 4. Representative regression properties.

Area	Mechanized checks
Generic cost	noncommutative `WordCost`; two-dimensional `StepsQueries`; additive measurement; value-preserving cost maps
Algebra/program	typed signature sum and dispatch; value-dependent joint value–cost sampling; input budgets; branch bounds; execution/support iff; first-order de Bruijn code with finite uniform sampling, exact budget, and assumption-free PPT construction
Oracle	adaptive exact trace; separated local/environment costs; total and per-name query bounds; noncommutative ordering; composed runtime
Machines/security	ordinary and dependent I/O through one core; structured operational realization; validated-code positive regression and negative pure/map/raw-record regressions; a DLog `Classical.choose` solver remains timed but cannot be auto-promoted; OTP perfect security; ElGamal correctness
UC kernel	FIFO order and SID isolation; typed routing capabilities; rejected send-as before corruption and authorized send-as afterward; dormant-address corruption and honest-state removal
UC composition	positive-fuel kernel simulation; common-fuel stability; context identity/associativity; `uc_compose`; actual FIFO activation of the layered broadcast manager

8 Soundness Boundaries and Future Work

The architecture makes several internal connections precise, but its claims remain relative to explicitly supplied models and certificates.

Higher-order program boundaries. `Program.Code` is higher-order. Values passed to `pure`, continuation functions in `bind`, and branch conditions are Lean terms, and their host reduction cost is not charged. All explicit primitive calls are measured, but neither a bounded higher-order program nor a value-only map is automatically promoted across the PPT boundary. The separate `Computation.FirstOrder` language uses typed de Bruijn environments and represented continuations, so a `FirstOrderOperationalCode` can be promoted internally. It is not a RAM or Turing-machine semantics: it currently has no recursion, loops, or concrete representation model.

Trusted annotations and observations. `CostedAlgebra.exec` is the authoritative exact annotation inside the formal model. The library proves that annotations compose and that bounds hold over their support; it does not prove that an annotation equals wall-clock CPU time. Likewise, `NatMeasure` is an explicit

Table 5. Mechanized guarantee versus instantiation obligation.

Mechanized guarantee	External or supplied obligation
Exact handlers compose costs once and in order	primitive annotations adequately model the intended implementation resource
Every supported path lies below a certified budget	the chosen abstract resource and budget are meaningful for the application
`NatMeasure` is monotone and additive	the observation is an adequate bridge from the abstract resource to runtime
Polynomial annotated runtime is proved for the exact stored run	canonical first-order code or an external backend relates that same run and runtime to a host-independent operational model
Oracle composition preserves values and proves a coarse bound	`CostExchange` and repeated-cost monotonicity hold when regrouping is requested
UC kernel steps are typed, charged, and FIFO	the concrete port schema, routing capabilities, policies, and component handlers model the intended execution
Certified fuel yields no-timeout and stability theorems	the supplied fuel certificate proves those semantic properties for the concrete world

monotone additive observation, not an automatically derived cost-model adequacy theorem.

Operational PPT admission. `PPTAdmissible`, `PPTOracleAdmissible`, and `PPTAddressedITMAdmissible` are transparent specializations of a common `OperationalRealization`. Such a realization exposes an operational model, one code object, and equations identifying its denotation and resource claim with the exact Lean artifact and claimed runtime. `OperationalModel.ValidCode` has an internal constructor only for the canonical first-order interpreter and structurally admitted primitive algebras. `PPTMachine.ofFirstOrderCode` therefore needs no opaque code-validity hypothesis. Other backends pass through the sole opaque `ExternalValidCode` relation; the generic library still provides no validation constructor for arbitrary Lean functions. Oracle composition and closed UC execution retain their separate operational obligations. This is an explicit soundness firewall, not a mechanized RAM-adequacy theorem: bottom algebra implementations and their declared costs must still be related to an intended concrete platform.

Worst-case paths only. `RandCosted.CostBound` is a support-wise worst-case predicate. The joint value–cost PMF remains available, but no expected-cost or tail-bound theory is currently built on it. Parallel composition, communication

rounds, encoded message length, and empirical host runtime are also outside the present core.

Oracle overapproximation and exchange. `PossibleExecution` intentionally forgets response support; only interpreter support implies a possible execution. Moreover, exact oracle interpretation itself needs no commutativity, but a coarse separated bound for interleaved caller and implementation work requires the explicit `CostExchange` hypothesis. Repeated-cost monotonicity is also stored explicitly because an arbitrary ordered additive monoid does not derive it from the order on natural counts alone.

Interactive certificates and capabilities. Fuel is solely a finite-execution control. The runner does not inspect a cost or fuel certificate and never removes a path because a proof is absent. `FuelCertificate.noTimeout` and stability are semantic proofs supplied for a concrete closed world. Port and send-as capabilities are only as strong as the concrete `PortSchema` and routing policy. UC security is therefore relative to the fixed address family, schema, cost model, measure, role wrappers, certificates, and operational admissions. In particular, `PPTAddressedITMCertificate` admits each role's exact handlers, while `PPTExecutionCertificate` separately admits the exact closed runner.

Current contexts use role-preserving injective address endomaps over a fixed global address and port-schema universe. They are not yet fully heterogeneous contexts translating arbitrary schemas. The layered bridge installs actual party, manager, boundary, and ideal-functionality components, but it does not by itself prove security of a concrete layered MPC protocol.

8.1 Next steps

The most direct extensions are: (i) add iteration and representation-level state to the minimal first-order language; (ii) relate selected resource algebras to a concrete RAM or machine semantics and discharge external backend obligations; (iii) add expected cost, tail bounds, parallel resources, communication, and round measures; (iv) generalize context transport across heterogeneous address and port schemas; (v) automate common fuel and operational-simulation proofs; (vi) instantiate the UC stack with concrete protocols and functionalities; and (vii) formalize the ElGamal IND-CPA reduction from DDH.

9 Conclusion

The current Crypto architecture treats exact execution as primary. Ordinary probability is cost erasure; a cost bound is a proposition over supported paths; a natural runtime is a monotone observation of an exact budget; polynomiality is the final annotation-level certificate; and an operational witness guards admission to the PPT interface. Typed primitive handlers, programs, oracle composition, dependent machines, and the UC kernel all follow this hierarchy.

The practical benefit is not merely uniform terminology. There is one place to inspect when checking the probability semantics of an algorithm, one exact source for each primitive cost, and an explicit proof chain from concrete paths to a polynomial annotated runtime, followed by a visible operational-admission path for the same run. Canonical first-order code discharges that path from reified syntax and structural primitive evidence; external backends retain a named trust obligation. Where an abstraction loses information—noncommutative oracle regrouping, environment-independent possible paths, finite fuel, or context transport—the required hypothesis is visible in the interface.

This organization establishes a coherent foundation for future cryptographic proofs while keeping its limits auditable. Extending the minimal first-order language with iteration and concrete representation costs, and instantiating the UC layer with protocols, can build on the same principle: exact execution first, erasure and certificates second, and host-independent admission only at an explicit boundary.

References

AHR+21. Carmine Abate, Philipp G. Haselwarter, Exequiel Rivas, Antoine Van Muylder, Théo Winterhalter, Catalin Hritcu, Kenji Maillard, and Bas Spitters. SSProve: A foundational framework for modular cryptographic proofs in coq. In Ralf Küsters and Dave Naumann, editors, *CSF 2021: IEEE 34th Computer Security Foundations Symposium*, pages 1–15, Virtual Conference, June 21–24, 2021. IEEE Computer Society Press. p. 3

Can01. Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *42nd Annual Symposium on Foundations of Computer Science*, pages 136–145, Las Vegas, NV, USA, October 14–17, 2001. IEEE Computer Society Press. p. 3

CSV19. Ran Canetti, Alley Stoughton, and Mayank Varia. EasyUC: Using EasyCrypt to mechanize proofs of universally composable security. In Stephanie Delaune and Limin Jia, editors, *CSF 2019: IEEE 32nd Computer Security Foundations Symposium*, pages 167–183, Hoboken, NJ, USA, June 25–28, 2019. IEEE Computer Society Press. p. 3

dMU21. Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021. p. 1

mC20. The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, pages 367–381. ACM, 2020. p. 1